

Week 7 Assignment

Name: Labhade Siddharth Popat

Student ID: CT_CSI_DE_1176

Dataset: Sample_Superstore

Assignment: Delta Lake MERGE Implementation

ScreenShots:

The screenshot displays the Databricks workspace interface. On the left, the navigation pane shows the 'Workspace' tab selected. The main area shows a notebook titled 'Assignment_7' with a Python code cell. The code reads a CSV file and loads it into a Delta table. Below the code, a table preview is shown with 11 rows of data.

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman	Consumer

Screenshot of the Databricks workspace interface showing a notebook titled "Assignment_7". The notebook is in Python mode and displays the output of a query: "4. Explore Dataset". The output shows a table with 5 rows and 9 columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, and Country. The table contains data for three customers: Claire Gute (Consumer), Darrin Van Huff (Corporate), and Sean O'Donnell (Consumer).

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	USA
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	USA
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	USA
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	USA
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	USA

Screenshot of the Databricks workspace interface showing a notebook titled "Assignment_7". The notebook is in Python mode and displays the output of a query: "df.printSchema()". The output shows the schema of the dataset, listing various columns and their data types, including Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, Sub-Category, Product Name, Sales, Quantity, Discount, and Profit.

```
df.printSchema()
-- Order ID: string (nullable = true)
-- Order Date: date (nullable = true)
-- Ship Date: date (nullable = true)
-- Ship Mode: string (nullable = true)
-- Customer ID: string (nullable = true)
-- Customer Name: string (nullable = true)
-- Segment: string (nullable = true)
-- Country: string (nullable = true)
-- City: string (nullable = true)
-- State: string (nullable = true)
-- Postal Code: integer (nullable = true)
-- Region: string (nullable = true)
-- Product ID: string (nullable = true)
-- Category: string (nullable = true)
-- Sub-Category: string (nullable = true)
-- Product Name: string (nullable = true)
-- Sales: string (nullable = true)
-- Quantity: string (nullable = true)
-- Discount: string (nullable = true)
-- Profit: double (nullable = true)
```


Screenshot of the Databricks workspace interface showing the notebook "Assignment_7". The notebook is titled "6.Remove Duplicate Records". The code in the notebook is as follows:

```
21
#Before removing duplicates
print("Rows Before:",df.count())

22
#Remove duplicates
df = df.dropDuplicates()

23
#After removing
print("Rows After:",df.count())
```

The notebook shows the results of the code execution: "Rows Before: 9994" and "Rows After: 9994". The notebook is running on a Serverless cluster. The interface includes a sidebar with navigation options like SQL, SQL Editor, Queries, Dashboards, and a top bar with search and user information.

Screenshot of the Databricks workspace interface showing the notebook "Assignment_7". The notebook is titled "7.Save as Delta Table". The code in the notebook is as follows:

```
25
# Rename columns to replace spaces with underscores for Delta compatibility
df_clean = df.select([col(c).alias(c.replace(" ", "_")) for c in df.columns])

# Save as Delta table
df_clean.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("superstore_delta")

27
delta_df = spark.read.format("delta").load("/tmp/superstore_delta")
```

The notebook shows the results of the code execution: "Rows After: 9994". The notebook is running on a Serverless cluster. The interface includes a sidebar with navigation options like SQL, SQL Editor, Queries, Dashboards, and a top bar with search and user information.

Screenshot of the Databricks workspace interface showing a notebook titled "8.Read Delta Table". The notebook is running a Python script that reads a Delta table named "superstore_delta". The output shows a table with 12 rows and 9 columns: Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Customer_Name, and Segment.

```
delta_df = spark.table("superstore_delta")
display(delta_df)
```

See performance (1) ✓ Checked for improvements

delta_df: pyspark.sql.connect.dataframe.DataFrame = [Row_ID: integer, Order_ID: string ... 19 more fields]

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	
1	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
2	17	CA-2014-105893	2014-11-11	2014-11-18	Standard Class	PK-19075	Pete Kirz	Consumer
3	21	CA-2014-143336	2014-08-27	2014-09-01	Second Class	ZD-21925	Zuschuss Donatelli	Consumer
4	53	CA-2015-115742	2015-04-18	2015-04-22	Standard Class	DP-13000	Darren Powers	Consumer
5	118	CA-2015-110457	2015-03-02	2015-03-06	Standard Class	DK-13090	Dave Kipp	Consumer
6	140	CA-2016-145583	2016-10-13	2016-10-19	Standard Class	LC-16885	Lena Creighton	Consumer
7	145	CA-2017-155376	2017-12-22	2017-12-27	Standard Class	SG-20080	Sandra Glassco	Consumer
8	147	CA-2014-110072	2014-10-22	2014-10-28	Standard Class	MG-17680	Maureen Gastineau	Home Office
9	186	CA-2016-105018	2016-11-28	2016-12-02	Standard Class	SK-19990	Sally Knutson	Consumer
10	200	US-2017-124303	2017-07-06	2017-07-13	Standard Class	FH-14365	Fred Hopkins	Corporate
11	237	CA-2017-160514	2017-11-12	2017-11-16	Standard Class	DB-13120	David Bremer	Corporate
12	246	CA-2014-131926	2014-06-01	2014-06-06	Second Class	DW-13480	Dianna Wilson	Home Office

Screenshot of the Databricks workspace interface showing a notebook titled "Assignment_7". The notebook is running a Python script that inserts a new record into a Delta table and then displays the updated table. The output shows a table with 2 rows and 9 columns: Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Customer_Name, and Segment.

```
# New record - will be INSERTED
insert_df = (
    delta_df.limit(1)
    .withColumn("Row_ID", lit(99999))
    .withColumn("Customer_Name", lit("Rahul Sharma"))
    .withColumn("City", lit("Mumbai"))
    .withColumn("State", lit("Maharashtra"))
)

# Combine both
increment_df = update_df.unionByName(insert_df)

display(increment_df)
```

See performance (1) ✓ Checked for improvements

update_df: pyspark.sql.connect.dataframe.DataFrame = [Row_ID: integer, Order_ID: string ... 19 more fields]

insert_df: pyspark.sql.connect.dataframe.DataFrame = [Row_ID: integer, Order_ID: string ... 19 more fields]

increment_df: pyspark.sql.connect.dataframe.DataFrame = [Row_ID: integer, Order_ID: string ... 19 more fields]

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	
1	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer
2	99999	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Rahul Sharma	Consumer

Siddharth-Labhade111/Celeb... Inbox (3) - siddharthlabhade11... LMS Assignment_7 - Databricks + Ask Gemini

dbc-83e260ed-f7e9.cloud.databricks.com/editor/notebooks/4918924784370817?o=7474656265518040#command/5589653082694102

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

+ New

Home Learn Workspace Recents Catalog Jobs & Pipelines Compute Discover Marketplace SQL SQL Editor Queries Dashboards Genie Agents Alerts Query History SQL Warehouses Data Engineering Runs Data Ingestion

Assignment_7

File Edit View Run Help Python Last edit was now

11.MERGE Operation

Run all Serverless Schedule Comments Share

```
from delta.tables import DeltaTable

deltaTable = DeltaTable.forName(spark, "superstore_delta")

deltaTable.alias("old") \
    .merge(
        increment_df.alias("new"),
        "old.Row_ID = new.Row_ID"
    ) \
    .whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()

print("MERGE Completed Successfully")
```

See performance (1) Optimize

MERGE Completed Successfully

24°C Partly cloudy

22:23 02-08-2026

Siddharth-Labhade111/Celeb... Inbox (3) - siddharthlabhade11... LMS Assignment_7 - Databricks + Ask Gemini

dbc-83e260ed-f7e9.cloud.databricks.com/editor/notebooks/4918924784370817?o=7474656265518040#command/5589653082694104

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

+ New

Home Learn Workspace Recents Catalog Jobs & Pipelines Compute Discover Marketplace SQL SQL Editor Queries Dashboards Genie Agents Alerts Query History SQL Warehouses Data Engineering Runs Data Ingestion

Assignment_7

File Edit View Run Help Python Last edit was now

12.Read Updated Delta Table

Run all Serverless Schedule Comments Share

```
final_df = spark.table("superstore_delta")

display(final_df)
```

See performance (1) Checked for improvements

final_df: pyspark.sql.connect.dataframe.DataFrame = [Row_ID: integer, Order_ID: string ... 19 more fields]

	Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment
1	21	CA-2014-143336	2014-08-27	2014-09-01	Second Class	ZD-21925	Zuschuss Donatelli	Consumer
2	53	CA-2015-115742	2015-04-18	2015-04-22	Standard Class	DP-13000	Darren Powers	Consumer
3	118	CA-2015-110457	2015-03-02	2015-03-06	Standard Class	DK-13090	Dave Kipp	Consumer
4	140	CA-2016-145583	2016-10-13	2016-10-19	Standard Class	LC-16885	Lena Creighton	Consumer
5	145	CA-2017-155376	2017-12-22	2017-12-27	Standard Class	SG-20080	Sandra Glassco	Consumer
6	147	CA-2014-110072	2014-10-22	2014-10-28	Standard Class	MG-17680	Maureen Gastineau	Home Office
7	186	CA-2016-105018	2016-11-28	2016-12-02	Standard Class	SK-19990	Sally Knutson	Consumer
8	200	US-2017-124303	2017-07-06	2017-07-13	Standard Class	FH-14365	Fred Hopkins	Corporate
9	237	CA-2017-160514	2017-11-12	2017-11-16	Standard Class	DB-13120	David Bremer	Corporate
10	246	CA-2014-131926	2014-06-01	2014-06-06	Second Class	DW-13480	Dianna Wilson	Home Office

24°C Partly cloudy

22:24 02-08-2026

Siddharth-Labhade111/Celebail x Inbox (3) - siddharthlabhade11 x LMS x Assignment_7 - Databricks x + Ask Gemini

dbc-83e260ed-f7e9.cloud.databricks.com/editor/notebooks/4918924784370817?o=7474656265518040#command/5589653082694108

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

Run all cells in this notebook.

Run all Serverless Schedule Comments Share

Assignment_7

File Edit View Run Help Python Last edit was now

13. Verify Total Number of Rows

Just now (1s) 38

```
print("Total Rows:", final_df.count())
```

See performance (1) ✓ Checked for improvements

Total Rows: 9995

14. Check Duplicate Records

Just now (1s) 40

```
from pyspark.sql.functions import count

final_df.groupBy("Row_ID") \
    .count() \
    .filter("count > 1") \
    .show()
```

See performance (1) ✓ Checked for improvements

Row_ID count

24°C Partly cloudy

22:26 02-08-2026

Siddharth-Labhade111/Celebail x Inbox (3) - siddharthlabhade11 x LMS x Assignment_7 - Databricks x + Ask Gemini

dbc-83e260ed-f7e9.cloud.databricks.com/editor/notebooks/4918924784370817?o=7474656265518040#command/5589653082694112

Search data, notebooks, recents, and more... CTRL + P

Verify identity workspace

Run all cells in this notebook.

Run all Serverless Schedule Comments Share

Assignment_7

File Edit View Run Help Python Last edit was now

15. Verify Updated Record

1 minute ago (1s) 42

```
final_df.filter(final_df.Row_ID == 99999).show(truncate=False)
```

See performance (1) ✓ Checked for improvements

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City	State	Postal_Code	Region
99999	CA-2014-105893	2014-11-11	2014-11-18	Standard Class	PK-19075	Rahul Sharma	Consumer	United States	Mumbai	Maharashtra	53711	Central

16. Verify Updated Existing Record

Just now (1s) 44

```
final_df.orderBy("Row_ID").show(5, False)
```

See performance (1) ✓ Checked for improvements

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City	State	Postal_Code

24°C Partly cloudy

22:29 02-08-2026

Siddharth-Labhade111/Celeb... x Inbox (3) - siddharthlabhade11... x LMS x Assignment_7 - Databricks x + Ask Gemini

dbc-83e260ed-f7e9.cloud.databricks.com/editor/notebooks/491892478437081?o=7474656265518040#command/5589653082694120

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

+ New Home Learn Workspace Recents Catalog Jobs & Pipelines Compute Discover Marketplace

SQL SQL Editor Queries Dashboards Genie Agents Alerts Query History SQL Warehouses

Data Engineering Runs Data Ingestion

File Edit View Run Help Python Last edit was now Run all Serverless Schedule Comments Share

only showing top 5 rows

17.Summary Statistics

2 minutes ago (1s) 46

```
final_df.describe().show()
```

See performance (1) ✓ Checked for improvements

summary	Row_ID	Order_ID	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City	State	Postal_Code
count	9995	9995	9995	9995	9995	9995	9995	9995	9995	9995
mean	5007.0049024512255	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	55190.23141570785
stddev	3037.4851835391905	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	32062.092577794214
min	1	CA-2014-100006	First Class	AA-10315	Aaron Bergman	Consumer	United States	Aberdeen	Alabama	1046
max	99999	US-2017-169551	Standard Class	ZD-21925	Zuschuss Donatelli	Home Office	United States	Yuma	Wyoming	99301

18.Display Final Dataset

Siddharth-Labhade111/Celeb... x Inbox (3) - siddharthlabhade11... x LMS x Assignment_7 - Databricks x + Ask Gemini

dbc-83e260ed-f7e9.cloud.databricks.com/editor/notebooks/491892478437081?o=7474656265518040#command/5589653082694120

Search data, notebooks, recents, and more... CTRL + P Verify identity workspace

+ New Home Learn Workspace Recents Catalog Jobs & Pipelines Compute Discover Marketplace

SQL SQL Editor Queries Dashboards Genie Agents Alerts Query History SQL Warehouses

Data Engineering Runs Data Ingestion

File Edit View Run Help Python Last edit was now Run all Serverless Schedule Comments Share

18.Display Final Dataset

1 minute ago (2s) 48

```
display(final_df)
```

See performance (1) ✓ Checked for improvements

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	
1	21	CA-2014-143336	2014-08-27	2014-09-01	Second Class	ZD-21925	Zuschuss Donatelli	Consumer
2	53	CA-2015-115742	2015-04-18	2015-04-22	Standard Class	DP-13000	Darren Powers	Consumer
3	118	CA-2015-110457	2015-03-02	2015-03-06	Standard Class	DK-13090	Dave Kipp	Consumer
4	140	CA-2016-145583	2016-10-13	2016-10-19	Standard Class	LC-16885	Lena Creighton	Consumer
5	145	CA-2017-155376	2017-12-22	2017-12-27	Standard Class	SG-20080	Sandra Glassco	Consumer
6	147	CA-2014-110072	2014-10-22	2014-10-28	Standard Class	MG-17680	Maureen Gastineau	Home Office
7	186	CA-2016-105018	2016-11-28	2016-12-02	Standard Class	SK-19990	Sally Knutson	Consumer
8	200	US-2017-124303	2017-07-06	2017-07-13	Standard Class	FH-14365	Fred Hopkins	Corporate
9	237	CA-2017-160514	2017-11-12	2017-11-16	Standard Class	DB-13120	David Bremer	Corporate
10	246	CA-2014-131926	2014-06-01	2014-06-06	Second Class	DW-13480	Dianna Wilson	Home Office
11	260	CA-2017-163139	2017-12-01	2017-12-03	Second Class	CC-12670	Craig Carreira	Consumer
12	263	US-2014-106992	2014-09-19	2014-09-21	Second Class	SB-20290	Sean Braxton	Corporate
13	267	CA-2017-136826	2017-06-16	2017-06-20	Standard Class	CB-12535	Claudia Bergmann	Corporate

Screenshot of the Databricks workspace interface showing the execution of a Python script to validate null values in a dataset.

The interface includes a sidebar with navigation options (Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion) and a top navigation bar with tabs for Assignment_7 and a search bar.

The main workspace area displays the following code and output:

```
from pyspark.sql.functions import *

final_df.select(
    count(when(col(c).isNull(), c)).alias(c)
    for c in final_df.columns
).show()
```

The output shows a table with columns: Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Customer_Name, Segment, Country, City, State, Postal_Code, Region, Product_ID, Category, Sub-Category. The values are mostly 0, indicating no nulls were found in the specified columns.

Below the code, the section "20. Validate Schema" is visible, showing the schema of the final DataFrame.

The bottom status bar indicates the temperature is 24°C and the weather is Partly cloudy.

Screenshot of the Databricks workspace interface showing the execution of a Python script to validate the schema of a dataset.

The interface includes a sidebar with navigation options (Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion) and a top navigation bar with tabs for Assignment_7 and a search bar.

The main workspace area displays the following code and output:

```
final_df.printSchema()

-- Order_ID: string (nullable = true)
-- Order_Date: date (nullable = true)
-- Ship_Date: date (nullable = true)
-- Ship_Mode: string (nullable = true)
-- Customer_ID: string (nullable = true)
-- Customer_Name: string (nullable = true)
-- Segment: string (nullable = true)
-- Country: string (nullable = true)
-- City: string (nullable = true)
-- State: string (nullable = true)
-- Postal_Code: integer (nullable = true)
-- Region: string (nullable = true)
-- Product_ID: string (nullable = true)
-- Category: string (nullable = true)
-- Sub-Category: string (nullable = true)
-- Product_Name: string (nullable = true)
-- Sales: string (nullable = true)
-- Quantity: string (nullable = true)
-- Discount: string (nullable = true)
-- Profit: double (nullable = true)
```

The output shows the schema of the final DataFrame, listing all columns and their data types and nullability.

The bottom status bar indicates the temperature is 24°C and the weather is Partly cloudy.